FloatingDraggableWidget (Flutter)

A reusable Flutter widget that allows **any child widget to be dragged freely on the screen**, while automatically staying within screen boundaries. Ideal for floating buttons, chat heads, mini-tools, or overlay UI elements.

✨Features

- Fully draggable widget using touch gestures
- Works with **any child widget**
- Configurable width & height
- Optional initial position support
- Automatically clamps position within screen bounds
- Lightweight and reusable

🍂Widget Overview

Widget type: `StatefulWidget`
Primary use case: Floating UI components (FAB alternatives, draggable cards, overlays)

🦏Constructor

```
const FloatingDraggableWidget({
  Key? key,
  required Widget child,
  double width = 120,
  double height = 120,
  Offset? initialPosition,
})
```

Parameters

Parameter	Type	Description
`child`	`Widget`	The widget to be displayed and dragged
`width`	`double`	Width of the draggable area (default: 120)
`height`	`double`	Height of the draggable area (default: 120)

Parameter	Type	Description
`initialPosition`	`Offset?`	Optional starting position on screen

🌹 Position State

```
late double xPosition;
late double yPosition;
```

- `xPosition` → Horizontal (left) offset
- `yPosition` → Vertical (top) offset

These values update continuously as the user drags the widget.

How It Works

1. Initial Position Logic

```
xPosition = widget.initialPosition?.dx ?? 100;
yPosition = widget.initialPosition?.dy ?? 100;
```

- Uses the provided `initialPosition` if available
- Falls back to a safe default position

---

2. Gesture Handling

```
onPanUpdate: (details) {
  setState(() {
    xPosition += details.delta.dx;
    yPosition += details.delta.dy;
  });
}
```

- `onPanUpdate` fires for every drag movement
- `details.delta` provides movement since last frame

---

3. Screen Boundary Clamping

```
xPosition = xPosition.clamp(0.0, screenSize.width - widget.width);
yPosition = yPosition.clamp(0.0, screenSize.height - widget.height);
```

Ensures: - Widget cannot move off-screen - Entire widget remains visible

---

⛈️Layout Structure

1. `Stack` → Allows free positioning
2. `Positioned` → Uses `xPosition` and `yPosition`
3. `GestureDetector` → Tracks drag gestures
4. `SizedBox` → Enforces widget dimensions
5. `child` → Your custom UI

🌳Usage Example

```
Scaffold(
  body: FloatingDraggableWidget(
    width: 80,
    height: 80,
    initialPosition: const Offset(200, 400),
    child: Container(
      decoration: BoxDecoration(
        color: Colors.blue,
        borderRadius: BorderRadius.circular(12),
      ),
      child: const Icon(Icons.chat, color: Colors.white),
    ),
  ),
);
```

🦗Notes & Limitations

- Must be placed inside a widget that allows overlays (e.g. `Scaffold` body)
- Clamp values depend on the provided `width` and `height`
- Frequent `setState` calls are expected during dragging

🦠 Customization Ideas

- Snap to screen edges on drag end
- Add drag inertia / fling animation
- Restrict dragging to X-axis or Y-axis
- Persist last position using `SharedPreferences`
- Convert to `AnimatedPositioned` for smooth motion

🦤 Full Source Code

```dart
import 'package:flutter/material.dart';

class FloatingDraggableWidget extends StatefulWidget {
  final double width;
  final double height;
  final Widget child;
  final Offset? initialPosition;

  const FloatingDraggableWidget({
    super.key,
    required this.child,
    this.width = 120,
    this.height = 120,
    this.initialPosition,
  });

  @override
  State<FloatingDraggableWidget> createState() =>
      _FloatingDraggableWidgetState();
}

class _FloatingDraggableWidgetState extends State<FloatingDraggableWidget> {
  late double xPosition;
  late double yPosition;

  @override
  void initState() {
    super.initState();
    // Use initial position if provided, otherwise default
    xPosition = widget.initialPosition?.dx ?? 100;
    yPosition = widget.initialPosition?.dy ?? 100;
  }

  @override
  Widget build(BuildContext context) {
```

```
    final screenSize = MediaQuery.of(context).size;

    return Stack(
      children: [
        Positioned(
          left: xPosition,
          top: yPosition,
          child: GestureDetector(
            onPanUpdate: (details) {
              setState(() {
                // update position and stay within screen bounds
                xPosition = (xPosition + details.delta.dx)
                    .clamp(0.0, screenSize.width - widget.width);
                yPosition = (yPosition + details.delta.dy)
                    .clamp(0.0, screenSize.height - widget.height);
              });
            },
            child: SizedBox(
              width: widget.width,
              height: widget.height,
              child: widget.child,
            ),
          ),
        ),
      ],
    );
  }
}
```